



PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL
FORMATO MATERIAL DE APOYO – ACTIVIDADES

MATERIAL DE APOYO APLICACIÓN WEB EN FLASK USANDO BASES DE DATOS MONGO DB

La aplicación ejemplo debe realizar un crud a una base de datos que contiene una colección llamada **PRODUCTOS** con los siguientes campos:

```
{  
    "código":valor,  
    "nombre":valor,  
    "precio":valor,  
    "categoría":valor  
}
```

Propuesta de Interfaz

GESTIÓN PRODUCTOS

Agregar					
Código	Nombre	Precio	Categoría	Foto	Acción
10	Televisor	2500000	ELECTRODOMESTICO		 
20	Camisa	120000	ROPA		 
50	Chaqueta	2000000	ROPA		 



Base de datos en MongoDB

A continuación se muestra la base de datos ya creada en **MONGO** mediante la herramienta **mongoDB Compass**

The screenshot shows the MongoDB Compass interface. On the left, the sidebar displays the connection to 'localhost:27017' and a list of databases including 'GESTIONPRODUCTOS'. The 'PRODUCTOS' collection is selected. The main panel shows the 'Documents' tab for the 'GESTIONPRODUCTOS.PRODUCTOS' collection. It includes a filter bar and two buttons: 'ADD DATA' and 'EXPORT COLLECTION'. Below, three document entries are displayed as JSON objects:

```
{
  "_id": ObjectId('641c43cf752e0be2785482eb'),
  "codigo": 10,
  "nombre": "Televisor",
  "precio": 2500000,
  "categoria": "ELECTRODOMESTICO"
}
```

```
{
  "_id": ObjectId('641c51966cdf307ddcb647eb'),
  "codigo": 20,
  "nombre": "Camisa",
  "precio": 120000,
  "categoria": "ROPA"
}
```

```
{
  "_id": ObjectId('641c79e0b0ce2a0211c23e02'),
  "codigo": 50,
  "nombre": "Chaqueta",
  "precio": 2000000,
  "categoria": "ROPA"
}
```

Nota: Las fotos de los productos se almacenan en la carpeta imágenes donde el nombre asignado será el id del producto y la extensión jpg.



Estructura de carpetas y archivos del Proyecto

```

✓ CRUD MONGODB
  > __pycache__
  ✓ controladores
    > __pycache__
    📄 controllerProducto.py
  ✓ static
    ✓ css
      # app.css
    ✓ imagenes
      🖼 641c43cf752e0be2785482eb.jpg
      🖼 641c79e0b0ce2a0211c23e02.jpg
      🖼 641c51966cdf307ddcb647eb.jpg
    ✓ js
      JS app.js
  ✓ templates
    <> encabezado.html
    <> frmAgregarProducto.html
    <> frmEditarProducto.html
    <> index.html
    <> listarProductos.html
    <> piePagina.html
  📄 app.py
```



Código archivo app.py

```
app.py > ...
1  from flask import Flask
2  import pymongo
3
4  #crear el objeto flask
5  app = Flask(__name__)
6  app.config["UPLOAD_FOLDER"]="./static/imagenes"
7
8  #crear la conexión a mongo
9  miConexion = pymongo.MongoClient("mongodb://localhost:27017")
10
11 #acceder a la base de datos
12 baseDatos = miConexion["GESTIONPRODUCTOS"]
13
14 #crear objet para referenciar la colección
15 productos = baseDatos["PRODUCTOS"]
16
17 from controladores.controllerProducto import *
18
19 if __name__=="__main__":
20     app.run(port=3000,debug=True)
```



Código archivo controllerProducto.py

Este archivo contiene las rutas y funciones que reciben las peticiones del cliente web. A continuación las librerías que se van a utilizar y la inclusión del objeto flask app y la colección productos creados en el archivo app.py. También se importa pymongo, la librería para cuando subimos archivos al servidor y la librería bson para poder convertir un valor en tipo de dato ObjectId.

```
controladores > controllerProducto.py > agregarProducto
1  from app import app, productos
2  from flask import Flask, request, render_template, redirect
3  import pymongo
4  from werkzeug.utils import secure_filename
5  import os
6  from bson.objectid import ObjectId
7
```

Al iniciar la aplicación, ella muestra la interfaz propuesta donde se lista en una tabla los productos que ya estén en la base de datos. A continuación se muestra el código de la ruta de la raíz de la aplicación.

```
8  @app.route("/")
9  def inicio():
10     #obtener la lista de productos de la colección productos
11     listaProductos=productos.find()
12     return render_template("listarProductos.html",
13                            listaProductos=listaProductos)
14
```



Código de la tabla html del archivo listarProductos.html

```
14 <table class="table table-bordered">
15   <thead class="bg-danger text-white text-center">
16     <tr>
17       <th>Código</th>
18       <th>Nombre</th>
19       <th>Precio</th>
20       <th>Categoría</th>
21       <th>Foto</th>
22       <th>Acción</th>
23     </tr>
24   </thead>
25   <tbody>
26     {% for producto in listaProductos %}
27       <tr>
28         <td>{{producto["codigo"]}}</td>
29         <td>{{producto["nombre"]}}</td>
30         <td>{{producto["precio"]}}</td>
31         <td>{{producto["categoria"]}}</td>
32         <td class="text-center">
33           
34         </td>
35         <td class="text-center" style="font-size:4vh">
36           <i class="fa fa-edit text-warning" title="Editar"></i>
37           <a href="/eliminar/{{producto['codigo']}}">
38             <i class="fa fa-trash text-danger" title="Eliminar"></i>
39           </a>
40         </td>
41       </tr>
42     {% endfor %}
43   </tbody>
44 </table>
```



Proceso Agregar Producto a la Base de Datos:

Código en el controlador

```
21 @app.route("/agregarProducto", methods=["POST"])
22 def agregarProducto():
23     mensaje=""
24     try:
25         #recibir los valores de la vista en variables locales
26         codigo=int(request.form["txtCodigo"])
27         nombre=request.form["txtNombre"]
28         precio=int(request.form["txtPrecio"])
29         categoria=request.form["cbCategoria"]
30
31         #datos de la imagen
32         archivo = request.files["fileFoto"]
33         nombreArchivo = secure_filename(archivo.filename)
34         listaNombreArchivo = nombreArchivo.rsplit(".",1)
35         extension = listaNombreArchivo[1].lower()
36
37         #crear el objeto producto de tipo diccionario
38         producto = {
39             "codigo":codigo,
40             "nombre":nombre,
41             "precio":precio,
42             "categoria":categoria
43         }
44
45         #validar si existe producto con código
46         existe=consultarProductoPorCodigo(codigo)
47         if(existe):
48             mensaje="Ya existe producto con ese código"
49             return render_template("frmAgregarProducto.html",
50                                     producto=producto,
51                                     mensaje=mensaje)
52         else:
53             #ejecutar consulta de inserción de datos
54             resultado= productos.insert_one(producto)
55             if resultado.acknowledged:
56                 mensaje="Producto Agregado Correctamente"
57                 #obtener el id del producto que se acaba de insertar
58                 idProducto = resultado.inserted_id
59                 nuevoNombre = str(idProducto) + "." + str(extension)
60                 archivo.save(os.path.join(app.config["UPLOAD_FOLDER"],nuevoNombre))
61                 return redirect("/")
62         except pymongo.errors as error:
63             mensaje=error
64             return render_template("frmAgregarProducto.html",
65                                     producto=producto,
66                                     mensaje=mensaje)
```



Función que valida si existe un producto de acuerdo a un determinado código en la base de datos.

```
68 def consultarProductoPorCodigo(codigo):
69     try:
70         consulta = {"codigo":codigo}
71         producto = productos.find_one(consulta)
72         if(producto is not None):
73             return True
74         else:
75             return False
76     except pymongo.errors as error:
77         print(error)
78         return False
```

Proceso Consultar Producto de acuerdo a su _id.

Inicialmente parte de la interfaz de listar productos cuando se da clic en icono de editar como lo señala la siguiente imagen.

GESTIÓN PRODUCTOS

Agregar					
Código	Nombre	Precio	Categoría	Foto	Acción
10	Televisor	\$2500000	ELECTRODOMESTICO		  
20	Camisa	\$120000	ROPA		 
50	Chaqueta	\$2000000	ROPA		 

El código html en el icono es el siguiente:

```
<a href="/consultar/{{producto['_id']}}">
|   <i class="fa fa-edit text-warning" title="Editar"></i>
</a>
```



Y el código **Python** en el controlador:

```
80 @app.route("/consultar/<string:idProducto>", methods=["GET"])
81 def consultarPorId(idProducto):
82     try:
83         idProducto=ObjectId(idProducto)
84         consulta = {"_id":idProducto}
85         producto = productos.find_one(consulta)
86         return render_template("frmEditarProducto.html",producto=producto)
87     except pymongo.errors as error:
88         mensaje=error
89         listaProductos =productos.find()
90         return render_template("listarProductos.html",
91                                mensaje=mensaje,
92                                listaProductos=listaProductos)
```

Dependiendo del producto seleccionando retorna a la siguiente vista:

GESTIÓN PRODUCTOS

EDITAR PRODUCTO

Código:
 ✓

Nombre:
 ✓

Precio:
 ✓

Categoría:
 ✓ ▼



Seleccione Foto(jpg) si va a cambiar:
 ✓



Nota: el formulario de editar, tiene un campo oculto llamado idProducto que recibe el campo _id de la colección.

```
<input type="hidden" name="idProducto" id="idProducto" value="{{producto['_id']}}">
```

Proceso actualizar: Parte de la interfaz anterior.

Código en el controlador Python

```
94 @app.route("/actualizar",methods=["POST"])
95 def actualizarProducto():
96     try:
97         #recibir los valores de la vista en variables locales
98         codigo=int(request.form["txtCodigo"])
99         nombre=request.form["txtNombre"]
100         precio=int(request.form["txtPrecio"])
101         categoria=request.form["cbCategoria"]
102         idProducto=ObjectId(request.form["idProducto"])
103         criterio = {"_id":idProducto}
104         datosActualizar = {
105             "codigo":codigo,
106             "nombre":nombre,
107             "precio":precio,
108             "categoria":categoria
109         }
110         consulta = {"$set": datosActualizar}
111         resultado=productos.update_one(criterio,consulta)
112
113         if(resultado.acknowledged):
114             mensaje="Producto Actualizado"
115             #verificar si viene foto para actualizarla
116             archivo = request.files["fileFoto"]
117             if(archivo.filename!=""):
118                 nombreArchivo = secure_filename(archivo.filename)
119                 listaNombreArchivo = nombreArchivo.rsplit(".",1)
120                 extension = listaNombreArchivo[1].lower()
121                 nombreArchivoActulizar = str(idProducto) + "." + str(extension)
122                 archivo.save(os.path.join(app.config["UPLOAD_FOLDER"],
123                                         nombreArchivoActulizar))
```



Proceso Eliminar:

Código Python

```
130 @app.route("/eliminar/<string:idProducto>" ,methods=["GET"])
131 def eliminarProducto(idProducto):
132     try:
133         idProducto = ObjectId(idProducto)
134         consulta = {"_id":idProducto}
135         resultado = productos.delete_one(consulta)
136         if(resultado.acknowledged):
137             mensaje="Producto eliminado"
138         else:
139             mensaje="Problemas al Eliminar"
140     except pymongo.errors as error:
141         mensaje = error
142     listaProductos=productos.find()
143     return render_template("listarProductos.html",
144                           mensaje=mensaje,listaProductos=listaProductos)
```

Código Javascript

```
2  function abrirModalEliminar(idProducto){
3      Swal.fire({
4          title: 'Eliminar Producto',
5          text: "¿Está seguro de eliminar?",
6          icon: 'warning',
7          showCancelButton: true,
8          confirmButtonColor: '#3085d6',
9          cancelButtonColor: '#d33',
10         cancelButtonText: 'NO',
11         confirmButtonText: 'SI'
12     }).then((result) => {
13         if (result.isConfirmed) {
14             location.href="/eliminar/"+idProducto
15         }
16     })
17 }
```



Código html en el icono eliminar:

```
<i onclick="abrirModalEliminar('{{producto["_id"]}}')"  
| class="fa fa-trash text-danger" title="Eliminar">  
</i>
```

Espero les sirva como material de apoyo.

CONTROL DEL DOCUMENTO

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	César Marino Cuéllar Chacón	Instructor	CIES-NEIVA	23-03-2023